

Universidad Politécnica de San Luis Potosí

**LAB: FILE PATH TRAVERSAL VALIDATION OF START OF
PATH**

Presentado por:

Christian Alfredo Morales Meza - 184937

Asignatura: Fundamentos de Ciberseguridad

Profesor: Servando López Contreras

Fecha de entrega: 09/09/2026

Índice de contenido

Índice de contenido	2
Ficha del laboratorio	3
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento paso a paso.....	3
Evidencia del procedimiento.....	4
Evidencias obligatorias	7
Explicación del payload.....	9
Mitigación recomendada.....	10

Ficha del laboratorio

Laboratorio: LAB: FILE PATH TRAVERSAL VALIDATION OF START OF PATH

Nivel: Practitioner

Tipo de explotación: File Path Traversal con evasión de validación de prefijo de ruta (Start of Path Validation Bypass)

Objetivo del laboratorio

Exfiltrar el contenido del archivo confidencial del sistema operativo `/etc/passwd` evadiendo un mecanismo de validación que exige que la ruta de entrada comience con un directorio base específico.

Explicación técnica de la vulnerabilidad

La causa raíz de esta vulnerabilidad radica en un diseño deficiente del mecanismo de validación de entradas en el backend de la aplicación. Para mitigar ataques de Path Traversal, los desarrolladores implementaron un filtro lógico que verifica si el valor del parámetro 'filename' inicia con el prefijo del directorio base legítimo (en este caso, `/var/www/images/`). Sin embargo, el error crítico a nivel de código es que el backend realiza esta validación de manera superficial (por ejemplo, mediante una comparación simple de cadenas de caracteres tipo 'startsWith') antes de resolver las secuencias de navegación de directorios de manera recursiva o sin aplicar una canonicalización previa del path. Al procesar la entrada posterior a la validación, la API de archivos del sistema operativo interpreta las secuencias de retroceso `../` de manera normal, permitiendo al atacante 'escalar' hacia arriba en el árbol de directorios partiendo desde el prefijo válido.

En términos de variantes técnicas, este escenario representa un 'Prefix-constrained Path Traversal' (Bypass de validación de inicio de ruta). Dentro del marco de OWASP Top 10, se clasifica rigurosamente en la categoría 'A01:2021-Broken Access Control'. El impacto técnico sobre la triada CIA se concentra de manera inmediata y crítica en la pérdida absoluta de Confidencialidad, permitiendo el acceso de lectura no autorizado a archivos sensibles del sistema de archivos del servidor (tales como credenciales de bases de datos, código fuente del aplicativo o configuraciones del sistema operativo como `/etc/passwd`). En un entorno de producción real, este tipo de acceso puede facilitar la exfiltración de información confidencial o encadenarse con otras vulnerabilidades para lograr la ejecución remota de comandos (RCE) mediante técnicas de log poisoning.

Procedimiento paso a paso

Fase de Reconocimiento e Interceptación de Tráfico:

Primero, se inicializa Burp Suite asegurando que la opción de interceptación en el módulo Proxy esté activa. Posteriormente, se accede al laboratorio mediante el navegador embebido de Burp y se explora el catálogo de productos de la tienda virtual. Al interactuar con el sitio, se observa que las imágenes de los productos se cargan bajo demanda. Al inspeccionar el tráfico capturado en la pestaña 'HTTP history', se identifica una serie de solicitudes dirigidas al endpoint `/image` que utilizan el parámetro de consulta `filename` con valores de ruta absoluta como

`/var/www/images/75.jpg`. Esto revela el directorio base utilizado por la aplicación para almacenar los recursos gráficos.

Fase de Análisis y Envío al Módulo Repeater:

Con el objetivo de realizar pruebas controladas sobre el parámetro sospechoso, se selecciona una petición GET típica de carga de imagen (específicamente la relacionada con '67.jpg') desde el historial de HTTP de Burp Suite. Haciendo clic derecho sobre la línea de la solicitud, se utiliza la opción 'Send to Repeater' (o el atajo de teclado Ctrl+R). En la pestaña del Repeater, se ejecuta un envío inicial ('Send') para verificar que la petición legítima responda con un código de estado HTTP 200 OK y devuelva correctamente los datos binarios del archivo de imagen JPEG.

Fase de Explotación y Evasión del Filtro:

Se procede a modificar el parámetro inyectando el payload de salto de directorio. Dado que el backend exige de manera estricta que la ruta comience con `/var/www/images/`, se mantiene dicho prefijo intacto y se concatenan inmediatamente después las secuencias de retroceso relativas necesarias junto con la ruta de destino deseada: `/var/www/images/../../../../etc/passwd`. Al dar clic en 'Send', se comprueba que el servidor web procesa la petición con éxito, evadiendo la regla de validación inicial y retornando el contenido íntegro del archivo `/etc/passwd`. Tras esta exfiltración exitosa, se refresca la página principal del navegador, confirmando que el laboratorio ha sido resuelto satisfactoriamente.

Evidencia del procedimiento

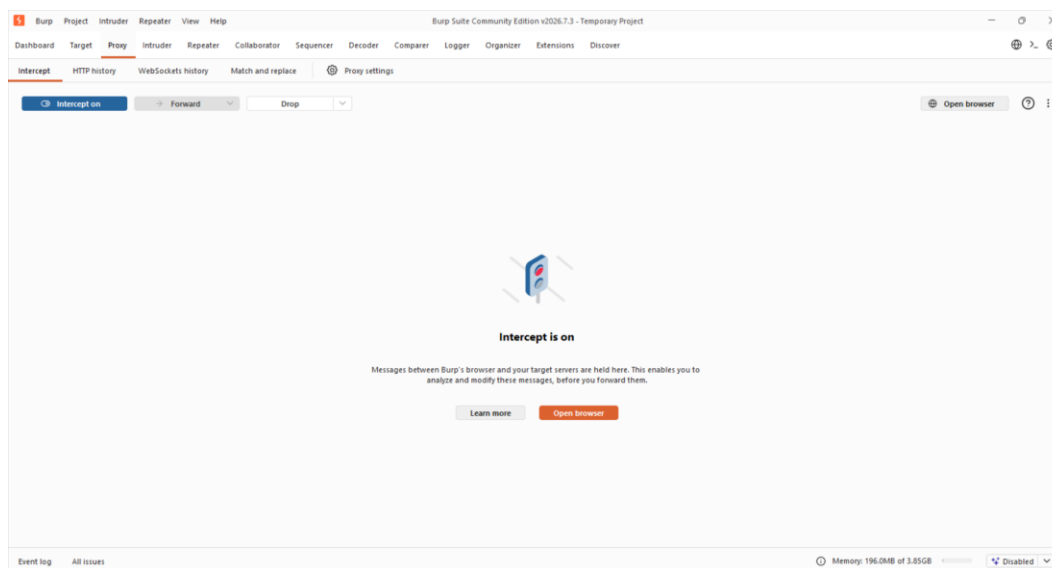


Figura 1. Activación de la herramienta de interceptación en la pestaña Proxy de Burp Suite para capturar el tráfico entrante y saliente.

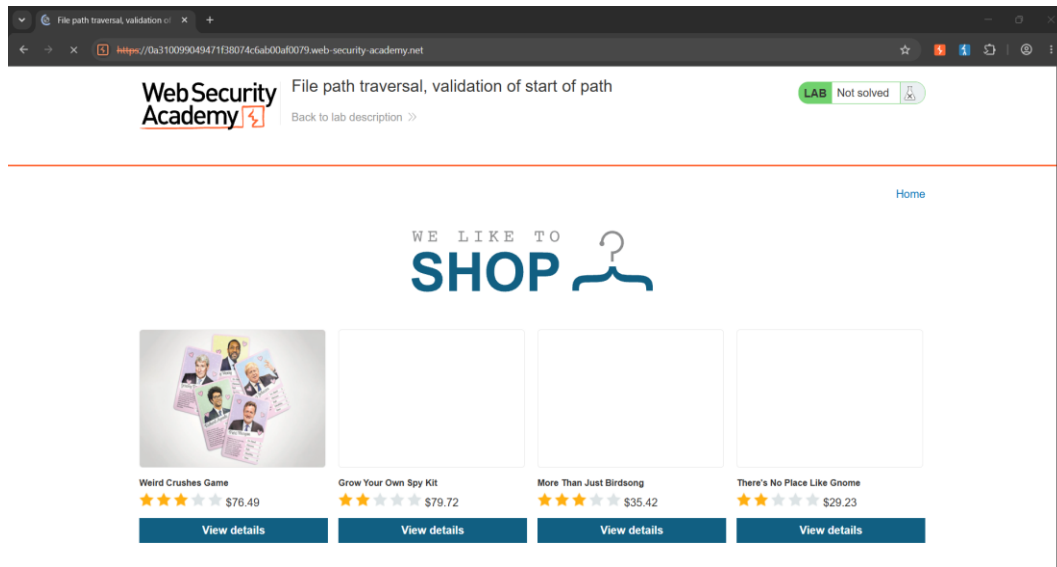


Figura 2. Acceso inicial a la página de inicio del laboratorio virtual, la cual muestra un catálogo de productos con imágenes cargadas dinámicamente.

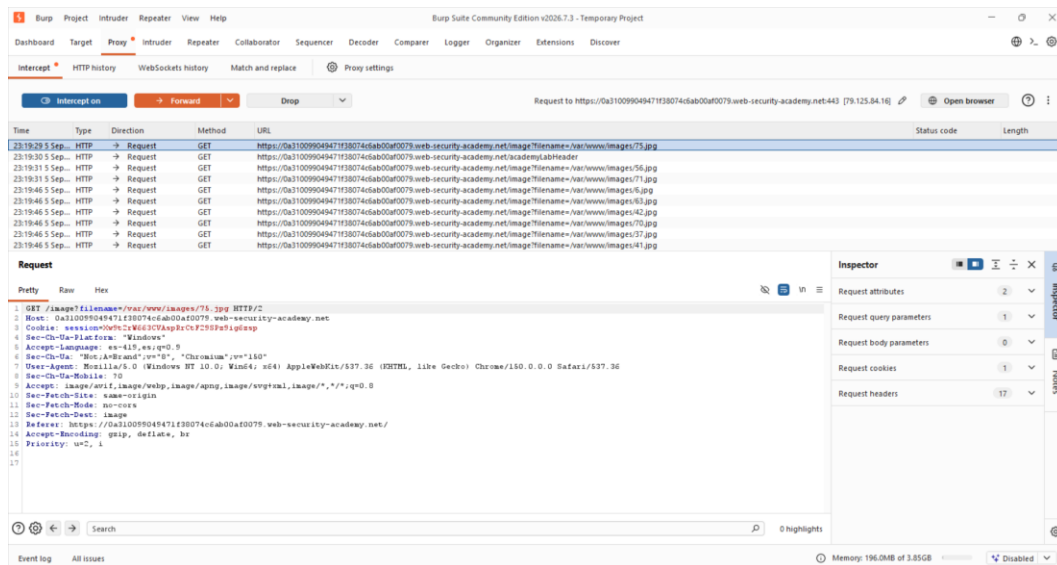


Figura 3. Interceptación de las peticiones GET dirigidas al endpoint `/image`, donde se revela el uso del parámetro `filename` con la ruta completa `/var/www/images/75.jpg`.

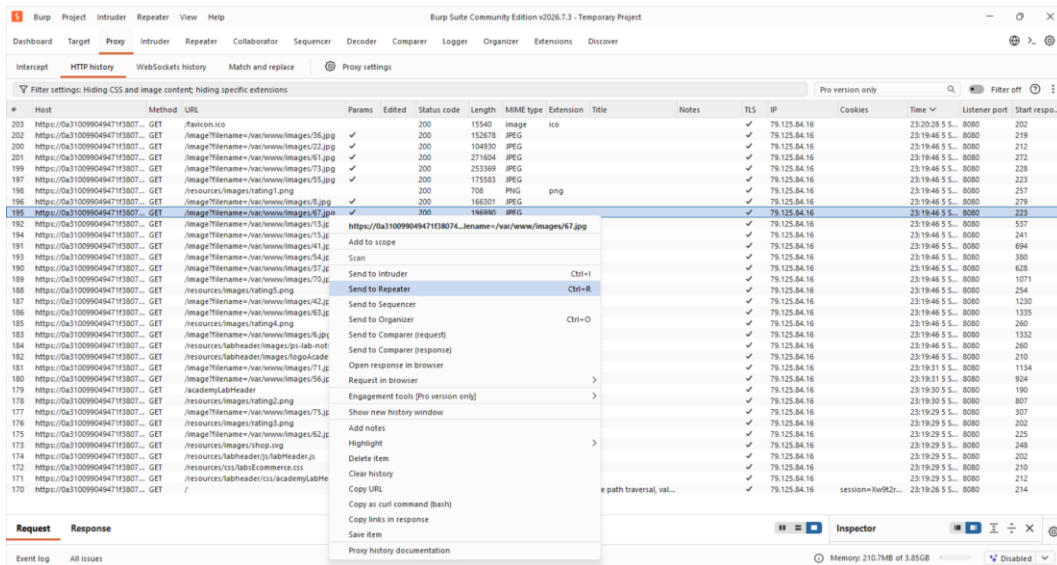


Figura 4. Selección de la petición HTTP correspondiente a la carga de la imagen número 67 desde el historial de navegación para enviarla al módulo Repeater.

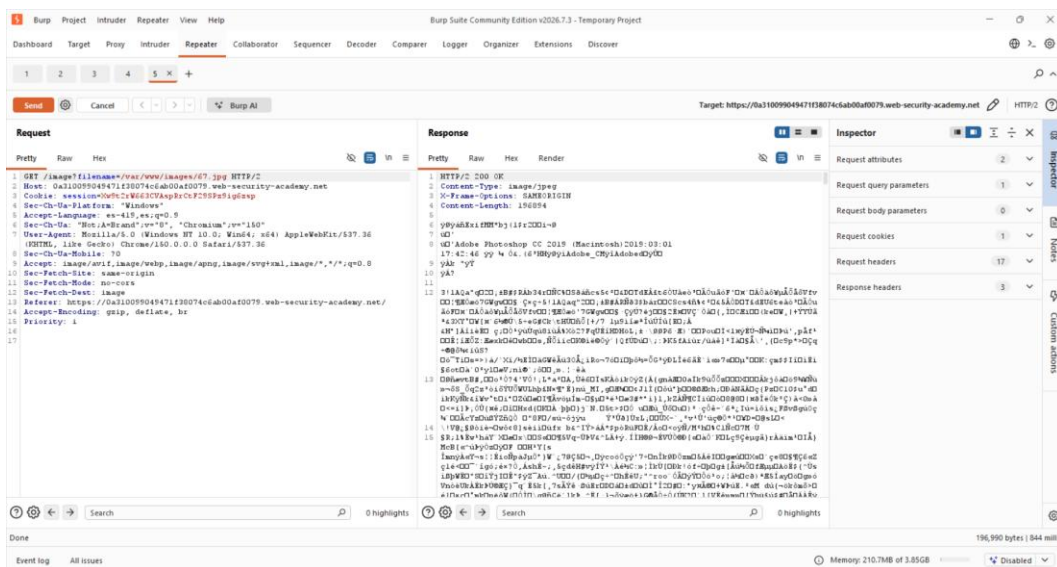


Figura 5. Envío inicial de la petición en Repeater para confirmar el comportamiento legítimo del servidor, el cual responde con un código HTTP 200 y el flujo binario de la imagen.

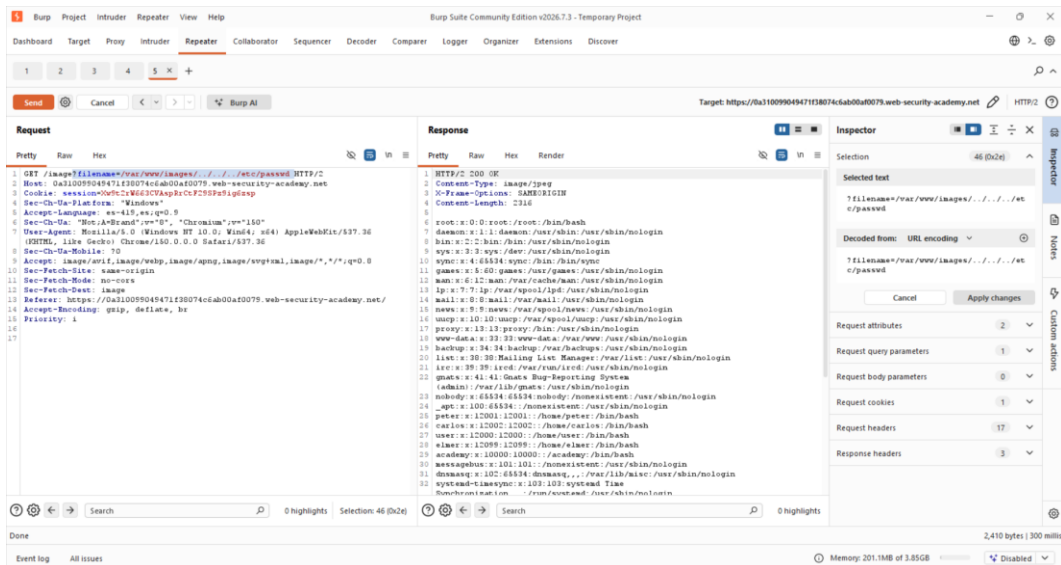


Figura 6. Modificación exitosa del parámetro `filename` inyectando secuencias de retroceso después de la ruta base permitida, logrando exfiltrar con éxito el archivo `/etc/passwd`.

Evidencias obligatorias

Request interceptado

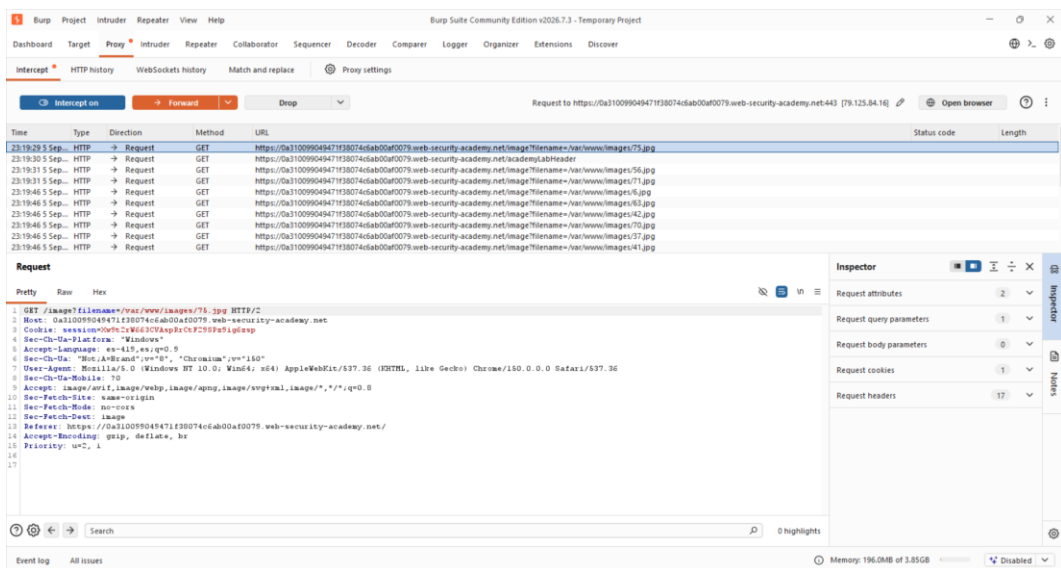


Figura 7. Request interceptado — Captura de pantalla de la petición GET original interceptada hacia `image`, revelando el parámetro `filename`.

Payload utilizado

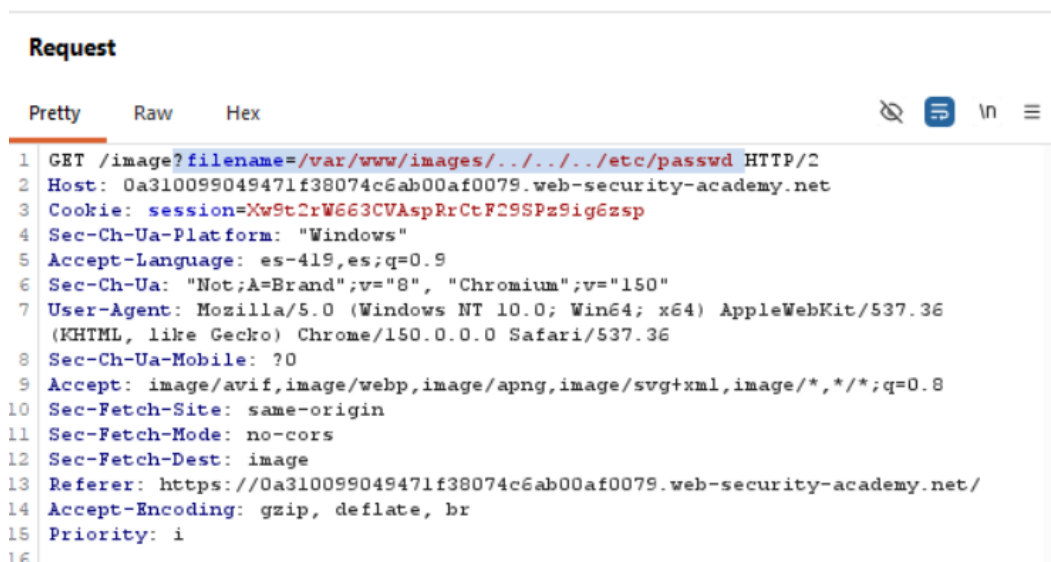


Figura 8. Payload utilizado — Inyección del payload ``var/www/images/../../../../etc/passwd`` en el parámetro ``filename`` dentro de la sección Request de Burp Suite Repeater.

Respuesta vulnerable

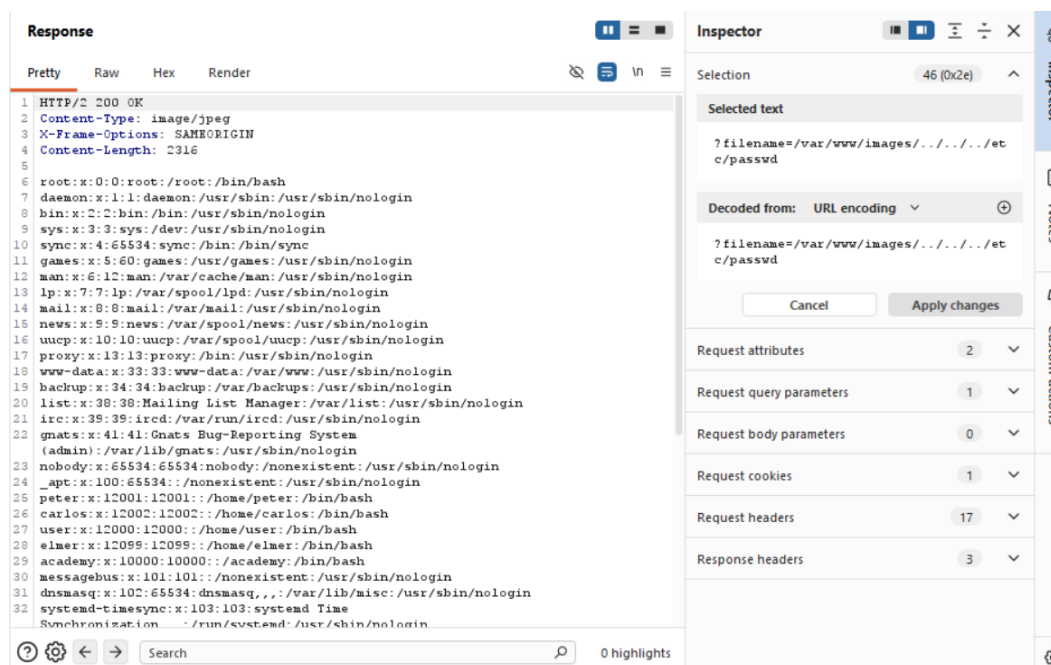


Figura 9. Respuesta vulnerable — Visualización de la respuesta del servidor con estado HTTP 200 OK, exponiendo las cuentas de usuario y configuraciones del sistema del archivo ``etc/passwd``.

Confirmación de “Lab Solved”

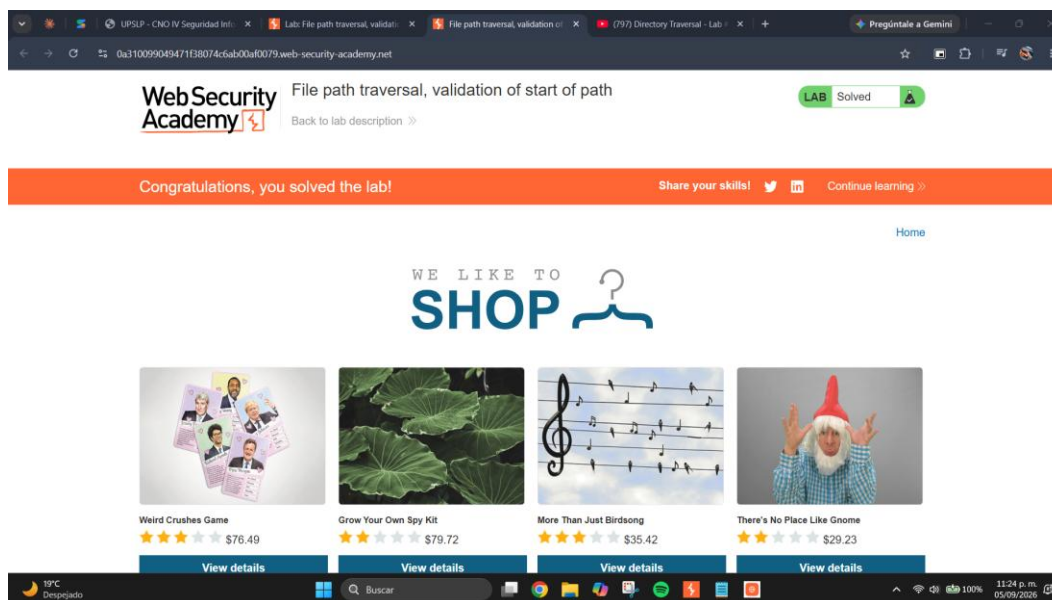


Figura 10. Confirmación de “Lab Solved” — Confirmación visual en la interfaz del navegador que muestra el banner de color naranja indicando 'Congratulations, you solved the lab!'.

Explicación del payload

El payload diseñado y utilizado para esta explotación es: ``/var/www/images/../../../../etc/passwd``.

Análisis sintáctico y lógico:

1. Prefijo obligatorio (``/var/www/images/``): Se coloca al principio del parámetro para satisfacer la validación lógica del lado del servidor que verifica de forma estricta el inicio de la cadena de texto de la ruta.
2. Secuencias de retroceso (``../../../../``): El carácter ``.` representa el directorio actual, mientras que la combinación ``..`` indica al sistema operativo que debe subir un nivel jerárquico en la estructura de carpetas. Al encadenar tres secuencias (``../../../../``), la ruta se desplaza de manera relativa hacia atrás tres niveles: de ``/var/www/images/`` sube a ``/var/www/``, luego a ``/var/`` y finalmente alcanza el directorio raíz del sistema (``/``).
3. Ruta del recurso objetivo (``etc/passwd``): Una vez posicionados en la raíz del sistema de archivos, el backend resuelve de forma continua el path descendiendo hacia el directorio ``etc`` para leer el archivo de texto estructurado ``passwd``.

Este payload aprovecha la falta de sanitización o de canonicalización previa a la validación de la entrada, permitiendo alterar por completo el flujo de control lógico de lectura de archivos de la aplicación.

Mitigación recomendada

Para asegurar una defensa en profundidad robusta contra vulnerabilidades de Path Traversal, se recomiendan las siguientes medidas correctivas:

1. Remediación en Código (Validación y Canonicalización):

- Evitar el paso directo de entradas provistas por el usuario a las APIs del sistema de archivos. En su lugar, es preferible utilizar identificadores indirectos indexados en una base de datos (por ejemplo, solicitar ``id=67`` y mapearlo internamente a un nombre de archivo seguro).
- Si es indispensable aceptar rutas de archivos, se debe realizar una 'canonicalización' de la ruta utilizando funciones del sistema de archivos nativas del lenguaje de programación (como ``getCanonicalPath()`` en Java, o ``realpath()`` en PHP) para resolver todas las secuencias relativas (``../``, enlaces simbólicos, codificaciones) antes de evaluar cualquier regla de seguridad. Posteriormente, se debe validar rigurosamente con una validación estricta que la ruta canonicalizada resultante empiece exactamente con la ruta del directorio base autorizado.

2. Controles de Arquitectura y Sistema Operativo:

- Aplicar el principio de menor privilegio (Least Privilege) configurando el proceso del servidor web (por ejemplo, ``www-data``, ``nobody`` o ``nginx``) para que se ejecute bajo un usuario con permisos mínimos de lectura en el sistema operativo, denegando el acceso total a rutas críticas como ``/etc/`` o archivos de configuración de otros servicios.
- Aislar la aplicación mediante el uso de contenedores (Docker, Podman) con sistemas de archivos de solo lectura y configuraciones de 'chroot jail' para limitar el alcance del sistema de archivos expuesto ante posibles fallas lógicas.
- Desplegar y configurar reglas específicas en un Web Application Firewall (WAF) para detectar, alertar y bloquear peticiones que contengan firmas conocidas de evasión de directorios.